



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Computação Gráfica

Ano Letivo de 2024/2025

Fase 1 - Primitivas gráficas

Grupo 5

Marco Soares Gonçalves (a104614) André Filipe Soares Pereira (a104275)
Salvador Duarte Magalhães Barreto(a104520) Leonardo Gomes Alves(a104093)

2 de Março de 2025

CG

Fase 1 - Primitivas gráficas

Grupo 5

Marco Soares Gonçalves (a104614) André Filipe Soares Pereira (a104275)

Salvador Duarte Magalhães Barreto(a104520) Leonardo Gomes Alves(a104093)

2 de Março de 2025

Índice

1	Introdução	1
2	Estrutura do projeto	2
3	Generator	4
3.1	Geração de plano	5
3.2	Geração de caixa	7
3.3	Geração de esfera	10
3.4	Geração de cone	13
4	Engine	17
4.1	Parser	17
4.2	Engine	17
5	Utilização	19
6	Conclusão	21

Lista de Figuras

3.1	Ficheiro cone_1_2_4_3.3d	4
3.2	Exemplificação para plano com $n2 = 3$	6
3.3	Ilustração dos planos definidos	8
3.4	Stack	10
3.5	Slice	11
3.6	Representação de uma esfera e os seus ângulos	11
3.7	Esfera com 10 slices e stacks	12
3.8	Base de um cone com 10 slices e 10 stacks	13
3.9	Cone no plano XZ	14
3.10	Cone no plano XY	15
3.11	Cone com 10 slices e 10 stacks	16
5.1	CMake	19
5.2	Make	20

1 Introdução

Este relatório documenta a primeira fase do desenvolvimento do projeto da disciplina de Computação Gráfica. O objetivo desta fase é criar um sistema capaz de gerar modelos geométricos e uma engine responsável por ler configurações em XML e renderizar essas primitivas.

O projeto é composto por duas aplicações distintas:

Generator: Responsável por gerar arquivos contendo informações sobre os vértices das primitivas gráficas especificadas.

Engine: Carrega um arquivo de configuração XML, interpreta os modelos gerados e renderiza a figura conforme os parâmetros fornecidos.

O formato dos arquivos é definido para armazenar as coordenadas dos vértices e outras informações necessárias para a correta reconstrução dos modelos na renderização. Para a leitura dos arquivos XML, utilizamos a biblioteca **TinyXML**, que permite a extração eficiente das informações da câmera e dos modelos a serem carregados.

Ao longo desta fase, a correta geração e visualização das primitivas gráficas foram verificadas comparando os resultados obtidos com as imagens de referência fornecidas. Este documento descreve a implementação do gerador e da engine, bem como as escolhas de estrutura de dados e formatos adotados.

2 Estrutura do projeto

A estrutura do projeto está organizada da seguinte forma:

build/: Pasta gerada pelo CMake, contém arquivos necessários para a compilação, o executável da engine e do gerador.

include/: Contém os cabeçalhos (.hpp) que definem as estruturas de dados do projeto. O design segue um modelo de herança, onde:

- **shape.hpp**: É a classe base (objeto) para todas as primitivas gráficas.
- **box.hpp, cone.hpp, plane.hpp, sphere.hpp**: Classes derivadas de shape.hpp, representando primitivas específicas para a caixa, cone, plano e esfera respetivamente.
- **vertex.hpp**: Define a estrutura para registar cada um dos vértices.
- **list.hpp**: Utilizado para armazenar listas de vértices ou modelos em memória, como requisitado no enunciado do projeto.
- **parser.hpp**: Estruturas usada para a leitura e interpretação dos ficheiros XML, assim como a leitura e interpretação dos ficheiros .3d, correspondente aos objetos.

objects/: Diretório que armazena os arquivos .3d gerados pelo generator, que contêm a lista de vértices da primitiva correspondente.

src/:

- **engine/**: Implementação do motor gráfico, responsável por carregar os arquivos .3d, interpretar o XML e renderizar os gráficos.
 - **engine.cpp**: Implementação principal do motor gráfico.
 - **parser.cpp**: Responsável por interpretar ficheiros XML e ficheiros .3d.
- **generator/**: Implementação do gerador de primitivas, responsável por criar os arquivos .3d.
 - **generator.cpp**: Função principal do gerador que chama funções específicas para cada primitiva.

- **box.cpp**, **cone.cpp**, **plane.cpp**, **sphere.cpp**: Contêm as funções que geram os pontos de cada estrutura associada: a caixa, o cone, o plano e a esfera, respectivamente.
- **vertex.cpp**: Auxilia no armazenamento e manipulação dos pontos.
- **tests/**: Pasta que guarda os arquivos XML usados para testar a engine, acompanhados de imagens de referência para validar a renderização.
- **tinyxml/**: Contém a biblioteca TinyXML, usada para interpretar os arquivos XML que definem a cena.

A organização segue um design modular; separamos claramente as responsabilidades entre geração de modelos, renderização e parsing de XML para facilitar a gestão e interpretação das responsabilidades de cada ficheiro no contexto da implementação.

3 Generator

O **generator** é a aplicação que gera de facto os ficheiros ".3d" requisitados, seguimos a sugestão definida no enunciado do projeto de incluir o número de vértices total no início do ficheiro. Cada três linhas do ficheiro descreve um triângulo através das coordenadas dos seus pontos, separados por vírgulas. Cada ponto está descrito com a sua coordenada nos eixos X,Y e Z. Na figura vemos o exemplo de um ficheiro "cone_1_2_4_3.3d" com o registo dos pontos do cone gerado associado:

```
> cone_1_2_4_3.3d
84
1.000000,0.000000,0.000000
0.666667,0.666667,0.000000
-0.000000,0.666667,0.666667
1.000000,0.000000,0.000000
-0.000000,0.666667,0.666667
-0.000000,0.000000,1.000000
-0.000000,0.000000,1.000000
-0.000000,0.666667,0.666667
-0.666667,0.666667,-0.000000
-0.000000,0.000000,1.000000
-0.666667,0.666667,-0.000000
-1.000000,0.000000,-0.000000
-1.000000,0.000000,-0.000000
-0.666667,0.666667,-0.000000
0.000000,0.666667,-0.666667
-1.000000,0.000000,-0.000000
0.000000,0.666667,-0.666667
0.000000,0.000000,-1.000000
0.000000,0.000000,-1.000000
0.000000,0.666667,-0.666667
0.666667,0.666667,0.000000
0.000000,0.000000,-1.000000
0.666667,0.666667,0.000000
1.000000,0.000000,0.000000
0.666667,0.666667,0.000000
0.333333,1.333333,0.000000
```

Figura 3.1: Ficheiro cone_1_2_4_3.3d

Para utilizar o **generator** é necessário um dos 4 comandos: "plane", "box", "sphere" ou "cone" que correspondem a um plano, caixa, esfera e cone. Cada um deles terá de receber valores numéricos associadas a características do objeto a gerar. Exemplos de utilização:

- ./generator plane
- ./generator box

- `./generator sphere`
- `./generator cone`

O "nome_ficheiro.3d" representa o ficheiro que será criado na pasta objects ao executar o comando. Seguem-se descrições completas relativas à implementação concreta da geração de cada uma das primitivas.

3.1 Geração de plano

Qualquer plano gerado tem que obedecer a regras impostas que são: ser um quadrado contido no plano XZ, estar centrado na origem e subdividido na direção X e Z.

Como mencionado previamente, os valores associados à geração deste objeto denominam-se "**length**"(**n1**) e "**divisions**"(**n2**), estes representam, respetivamente, o valor do lado do quadrado a desenhar e o número de divisões do mesmo, ou seja, quantas vezes deve estar subdividido nas direções requisitadas. O plano gerado será uma junção de n^2 **quadrados** que são eles mesmos formados por 2 triângulos; cada um destes quadrados terá um lado de comprimento igual a $n1/n2$.

Para gerar o nosso plano desenvolvemos um ciclo que percorre o eixo X e um ciclo que percorre o eixo Z, ou seja, determinamos os pontos de todos os triângulos dentro da área relevante ao implementar as seguintes formulas a cada iteração (passos de tamanho $n1/n2$):

- Pontos:
 - $x1 = x_inicial + passo_X * (n1/n2)$
 - $z1 = z_inicial + passo_Z * (n1/n2)$
 - $x2 = x1 + (n1/n2)$
 - $z2 = z1 + (n1/n2)$
- Triângulo 1
 - Ponto 1- $x1, 0, z1$
 - Ponto 2- $x1, 0, z2$
 - Ponto 3- $x2, 0, z2$
- Triângulo 2
 - Ponto 1- $x2, 0, z1$

- Ponto 2- $x_1, 0, z_1$
- Ponto 3- $x_2, 0, z_2$

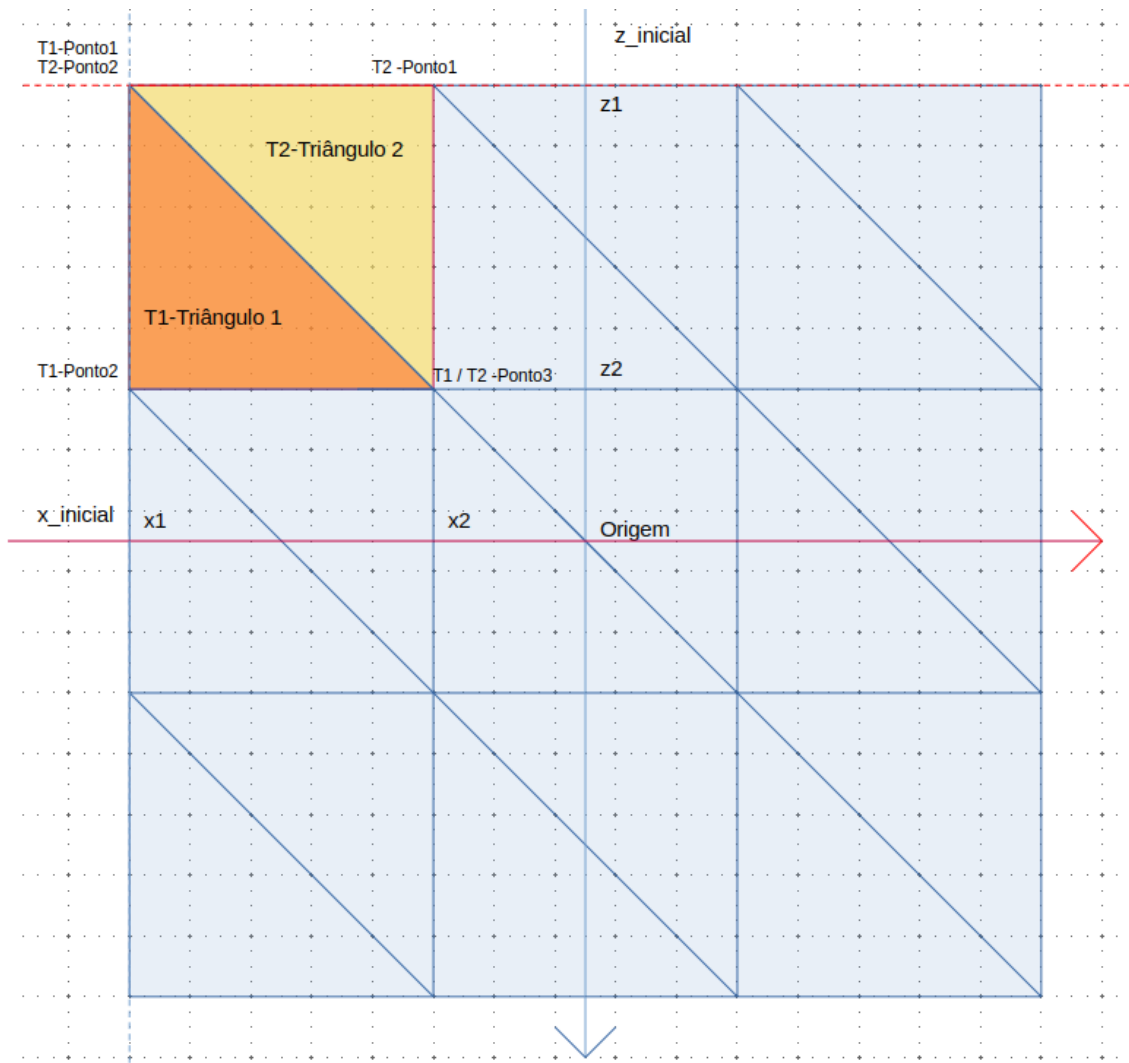
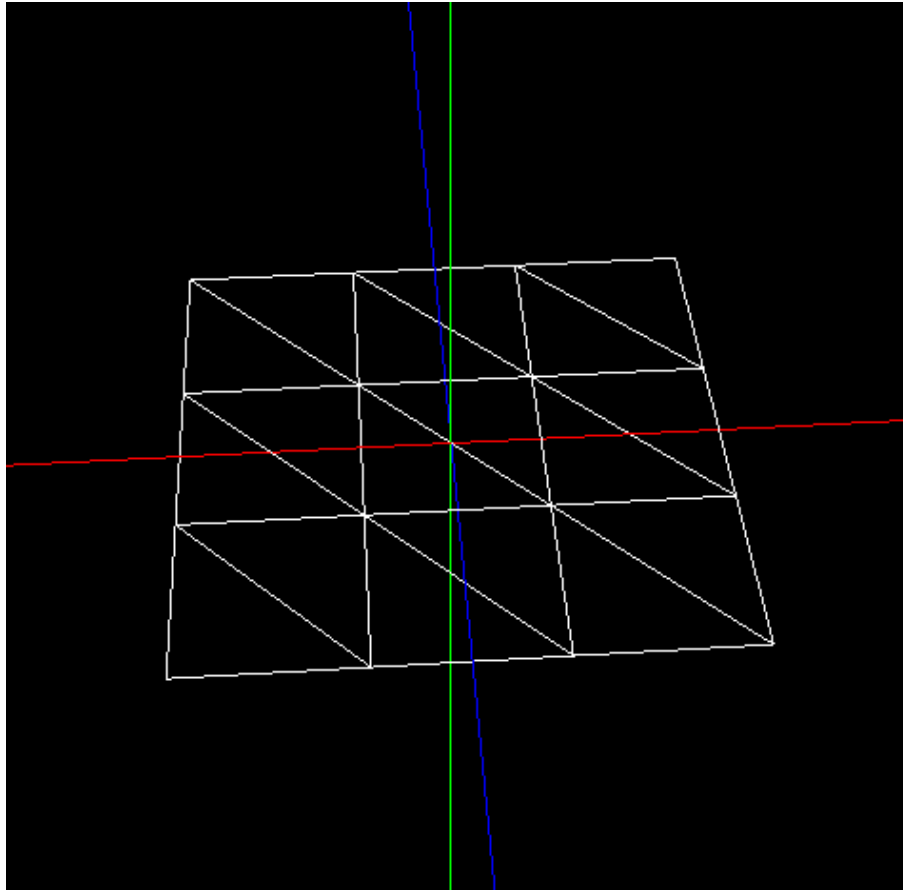


Figura 3.2: Exemplificação para plano com $n_2 = 3$

Na imagem acima, o lado do plano terá comprimento igual a n_1 e necessitaremos de 3×3 iterações para desenhar os 9 quadrados dos quais o plano consiste. Segue-se este mesmo exemplo renderizado através do ficheiro .3d gerado:



3.2 Geração de caixa

De forma idêntica ao plano, a caixa recebe como argumentos valores para **"length"(n1)** e **"divisions"(n2)**, para além disso fica centrada na origem.

A caixa resume-se à junção de 6 planos, 4 planos laterais 1 superior e 1 inferior. Para desenhar todos os planos nas posições corretas foi necessário que fossem paralelos a planos definidos pelos eixos XYZ; deste modo os planos de "cima" e "baixo" são paralelos a XZ, os de "frente" e de "trás" a XY e os planos laterais "esquerda" e "direita" a YZ.

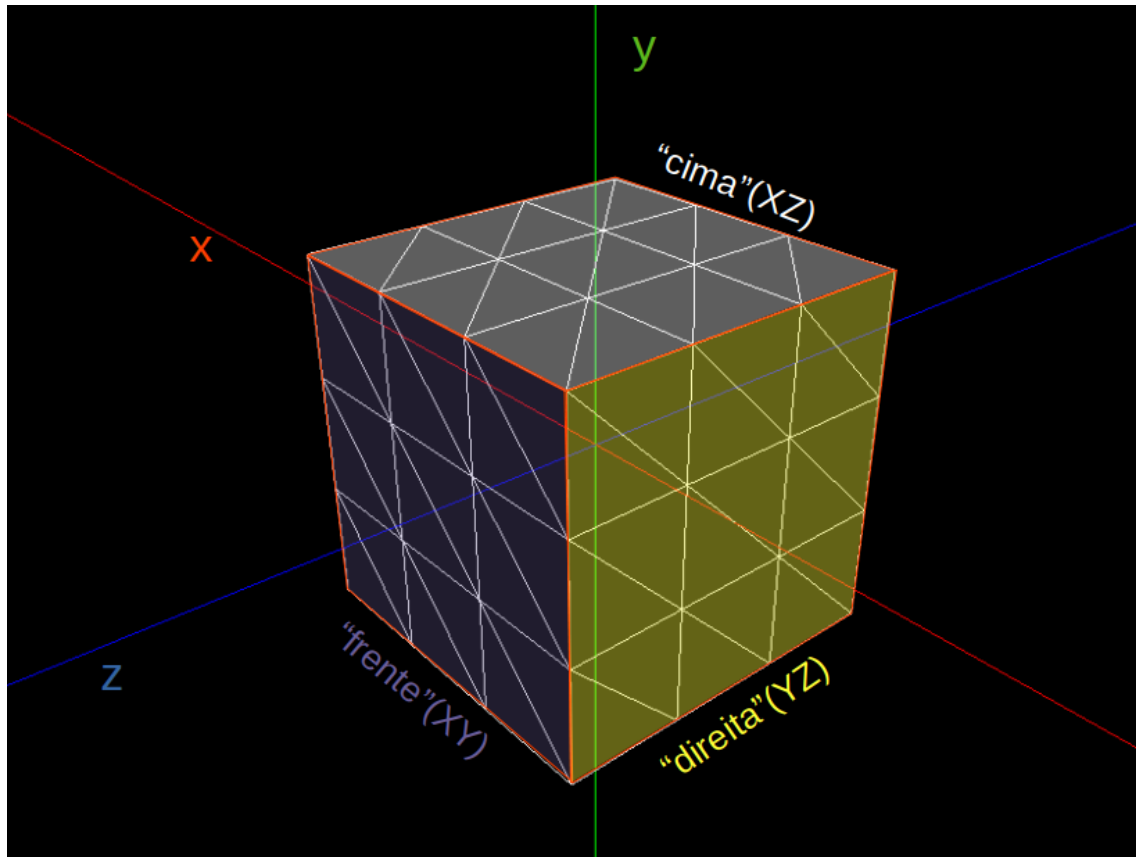


Figura 3.3: Ilustração dos planos definidos

O processo de iteração no desenho de cada plano individual foi descrito anteriormente, agora vamos definir os pontos 1-4 utilizados para definir os limites de cada quadrado e que, em conjunto, formam a caixa:

Plano XZ:

Aqui pretendemos criar o plano inferior e superior da caixa logo os pontos serão idênticos nas suas coordenadas exceto no valor "altura" que será $-n1/2$ para o plano inferior e $n1/2$ para o plano superior:

- Ponto1- $(-n1/2, altura, -n1/2)$
- Ponto2- $(n1/2, altura, -n1/2)$
- Ponto3- $(-n1/2, altura, n1/2)$
- Ponto4- $(n1/2, altura, n1/2)$

Plano XY:

Paralelos a este plano são os planos de frente e trás da nossa caixa; agora será o valor no eixo Z que varia para os planos com os pontos que definem o quadrado idênticos para ambos os planos, exceto para essa coordenada.

- Ponto1- $(-n1/2, n1/2, valor_z)$
- Ponto2- $(n1/2, n1/2, valor_z)$
- Ponto3- $(-n1/2, -n1/2, valor_z)$
- Ponto4- $(n1/2, -n1/2, valor_z)$

Plano YZ:

Finalmente o plano YZ foi utilizado para definir os planos da esquerda e direita da caixa onde o valor de X será aquele que distingue a variação de posicionamento entre eles.

- Ponto1- $(valor_x, n1/2, n1/2)$
- Ponto2- $(valor_x, n1/2, -n1/2)$
- Ponto3- $(valor_x, -n1/2, n1/2)$
- Ponto4- $(valor_x, -n1/2, -n1/2)$

Os valores "valor_z" e "valor_x" podem tomar apenas valor $n1/2$ ou $-n1/2$, tal como a "altura".

É importante realçar que, para definir dois planos paralelos, devemos ter em atenção a forma que definimos os pontos, isto é, anti-clockwise dependendo da sua posição; garantindo a correta renderização dos planos.

3.3 Geração de esfera

Para conseguir gerar uma esfera precisamos 3 valores: **"radius"(n1)**, **"slices"(n2)** e **"stacks"(n3)**.

Tivemos que traduzir as coordenadas esféricas para coordenadas cartesianas para realizar a implementação, seguindo assim algumas das dicas que retemos das aulas práticas.

- $x = n1 \times \cos \beta \times \sin \alpha$
- $y = n1 \times \sin \beta$
- $z = n1 \times \cos \beta \times \cos \alpha$

Onde α corresponde ao ângulo da longitude, tendo valores $[0, 2\pi]$ e β corresponde ao ângulo da latitude, tendo valores $[-\frac{\pi}{2}, \frac{\pi}{2}]$.

Os valores de "slices", definem o número de "fatias" que constituem a esfera, ou seja, as divisões verticais da mesma, enquanto que as "stacks" definem o número de divisões horizontais.

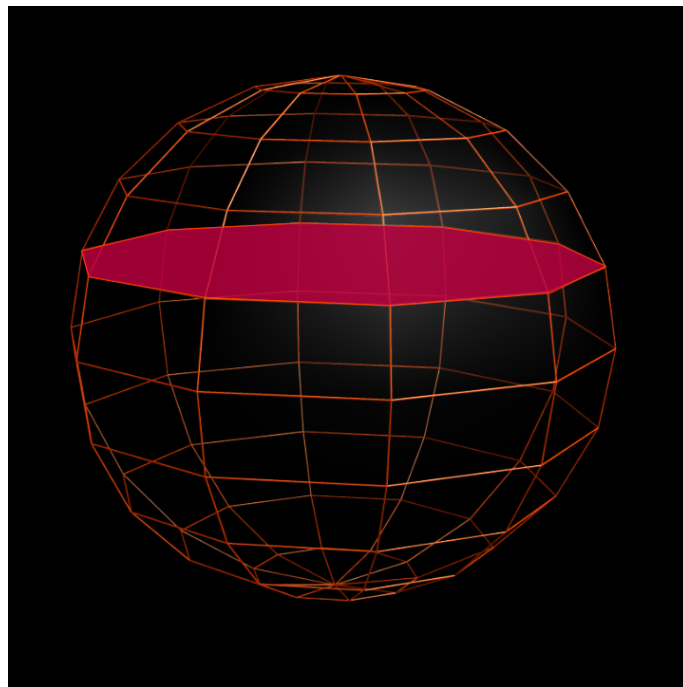


Figura 3.4: Stack

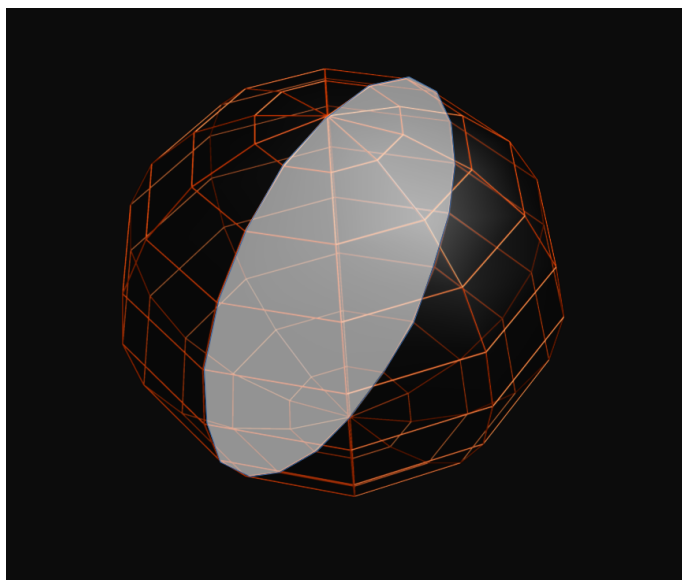


Figura 3.5: Slice

Com isto em mente, começamos por determinar qual seria o valor de **delta alfa**, ou seja, o valor com o qual nos conseguimos "deslocar" de uma slice para outra ao somar delta alfa ao valor atual de alfa; recalculas as coordenadas com este novo valor deve permitir obter as coordenadas da próxima "fatia"; e o mesmo para as stacks, ao somar um **delta beta** ao valor do ângulo beta.

Desta forma, os valores destes deltas foram calculados da seguinte forma:

- $\Delta \text{Alfa} = \frac{2\pi}{n2(\text{slices})}$
- $\Delta \text{Beta} = \frac{\pi}{n3(\text{stacks})}$

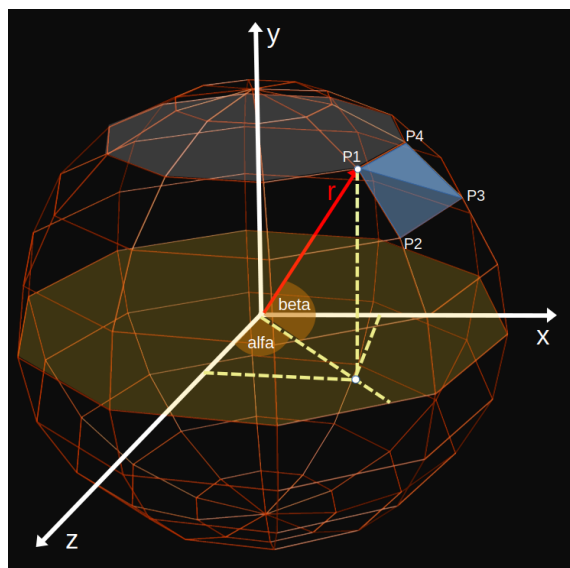


Figura 3.6: Representação de uma esfera e os seus ângulos

Para determinar as coordenadas de cada um dos pontos da esfera, percorremos todas as combinações possíveis dos valores α e β , ao incrementar com os seus respectivos valores de delta.

Cada par de valores α e β é capaz de representar um ponto na superfície esférica.

A interseção das slices com as stacks forma quadrados que podemos desenhar com dois triângulos, como podemos observar no quadrado definido pelos pontos P1, P2, P3 e P4 na imagem anterior.

As diferenças que existem do ponto P1 para os restantes são:

- **P1 - P2:** possuem o mesmo valor de α mas os valores de β diferem;
- **P1 - P3,** os valores de α e β diferem;
- **P1 - P4,** os valores de α são diferentes, mas possuem o mesmo valor de β .

Assim, a tradução para código destes conceitos será percorrer todas as slices de cada uma das stacks e, através dos valores de α e β representar os quadrados, formados por triângulos.

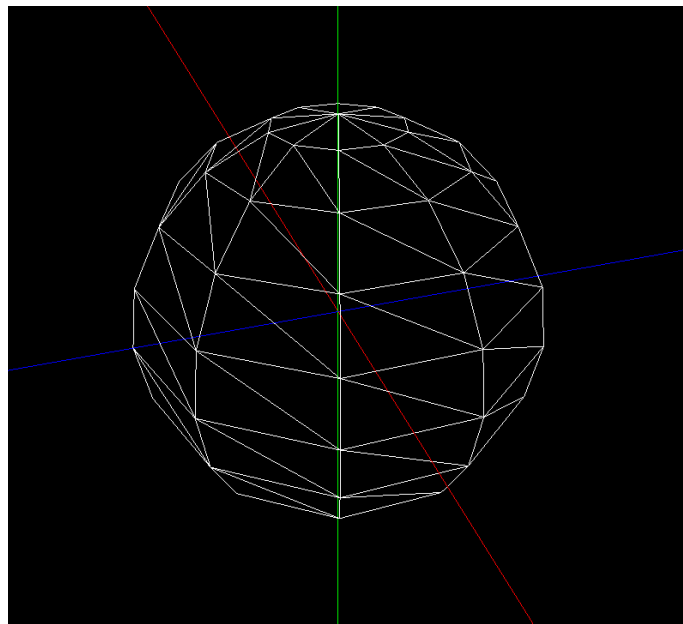


Figura 3.7: Esfera com 10 slices e stacks

3.4 Geração de cone

Para conseguirmos desenhar o cone, decidimos utilizar uma estratégia parecida com a da esfera uma vez que, o cone também é constituído por **slices** e **stacks**.

Assim, começamos por desenhar a base do cone, que será paralelo ao plano XZ. Para tal, sabemos que a base é composta por um certo número de slices, por isso, começamos por determinar a amplitude do ângulo delta alfa, que servirá de "step", para auxiliar no cálculo dos pontos seguintes. Este valor é determinado pelo cálculo de: $2\pi / \text{slices}$.

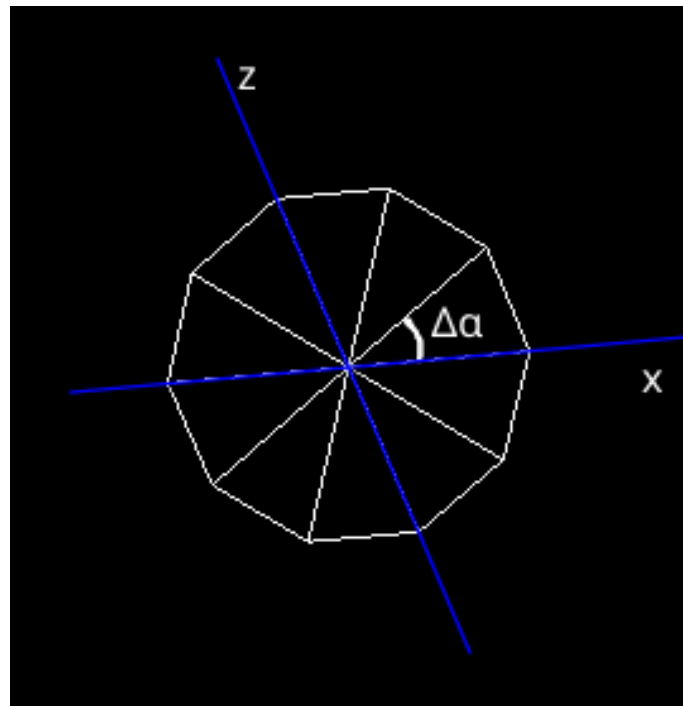


Figura 3.8: Base de um cone com 10 slices e 10 stacks

Assim, como podemos verificar na imagem acima, o valor de delta alfa auxilia-nos a calcular as coordenadas dos pontos, no caso as coordenadas calculam-se da seguinte maneira:

- $X - \text{raio} \times \cos \alpha$;
- $Y - 0$ pois a base encontra-se no plano XZ;
- $Z - \text{raio} \times \sin \alpha$.

É importante realçar novamente que, na escrita do código, devemos ter em atenção a ordem com que definimos os pontos, uma vez que a base apenas deve ser vista com a câmara numa posição com Y menor 0.

Com a base feita, decidimos começar por descobrir uma forma de encontrar as coordenadas X e Z dos pontos que iriam definir as faces laterais do cone.

Os valores da coordenada Y são facilmente calculados, uma vez que:

$$\text{altura de uma stack} = \frac{\text{altura do cone}}{\text{número de stacks}}$$

Com isto em mente, uma vez que os pontos que agora precisamos de calcular são os pontos X e Z, decidimos representar um cone com 2 stacks (visto dum ponto onde Y é positivo) no plano XZ, obtendo assim o seguinte:

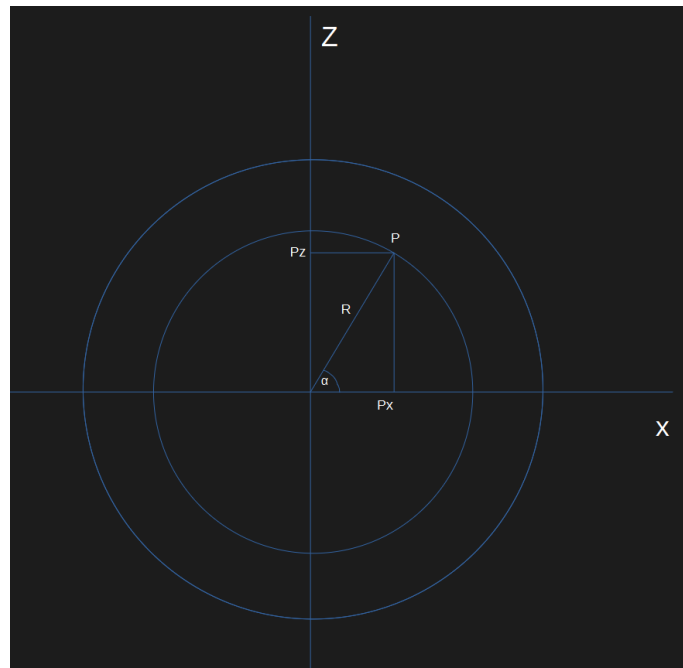


Figura 3.9: Cone no plano XZ

Apesar de a imagem não estar na escala correta, somos capazes de determinar quais queres pontos (coordenadas X e Z) de uma stack, necessitando apenas do ângulo alpha ($\alpha = 0$ encontra-se no eixo positivo do X) e do valor da reta R. O valor de alfa é trivial, uma vez que este irá variar de 0 até 2π , assim como a base do cone e, dependendo das slices, teremos um delta alpha que já foi calculado, $\frac{2\pi}{\text{slices}}$.

Observando então o ponto P presente na segunda stack, as suas coordenadas X e Z são calculados da seguinte forma:

- $X = R \times \cos(\alpha)$
- $Z = R \times \sin(\alpha)$

Assim, basta-nos apenas determinar o valor de R, que é a distância do ponto central da base a um dado ponto do cone.

Como podemos ver na imagem abaixo, o cone visto no plano XY é um triângulo e, como a sua altura é dividida por stacks, é possível encontrar uma fórmula que determine este valor de R.

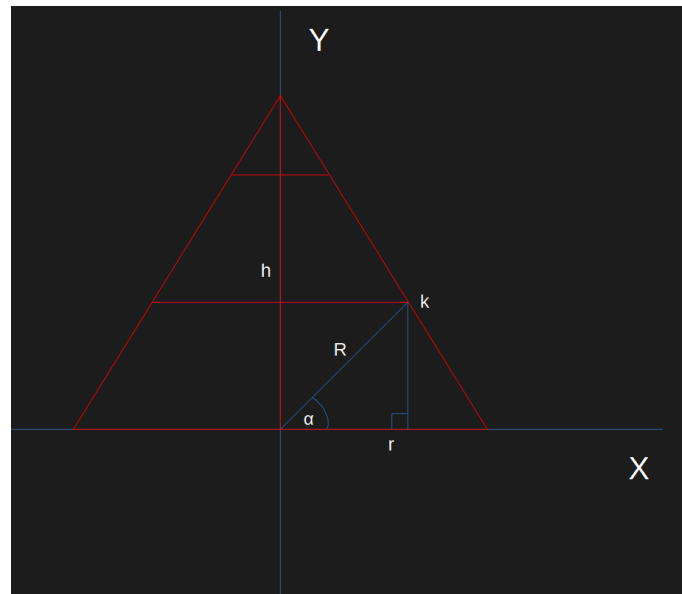


Figura 3.10: Cone no plano XY

Por observação, conseguimos reparar que podemos aplicar o Teorema de Tales a este triângulo, uma vez que as retas das stacks são paralelas entre si e são interseccionadas por duas retas transversais, reta da altura e a reta lateral do cone.

Desta forma, considerando r como o raio do cone, h como a sua altura, R como o valor que pretendemos saber, e k como altura dessa stack sendo então no teorema de Tales traduzido para h - k, conseguimos obter a seguinte igualdade:

$$\frac{r}{R} = \frac{h}{h - k}$$

Desta forma, basta apenas determinarmos o valor de k, para conseguirmos formular o valor de R, dada uma certa stack.

$$k_i = \frac{i}{\text{stacks}} \times h, \text{ sendo } i = 0 \text{ a stack com } y = 0$$

Sabendo então a fórmula de k, basta substituir essa variável no teorema e resolver em ordem a R, obtendo então a seguinte fórmula para R:

$$R_i = r \times \left(1 - \frac{i}{\text{stacks}}\right)$$

Com esta fórmula, conseguimos finalmente obter o valor de R , sendo possível assim calcular qualquer ponto do cone, tendo sempre em atenção às slices e stacks, bastando agora aplicar estas fórmulas no nosso código.

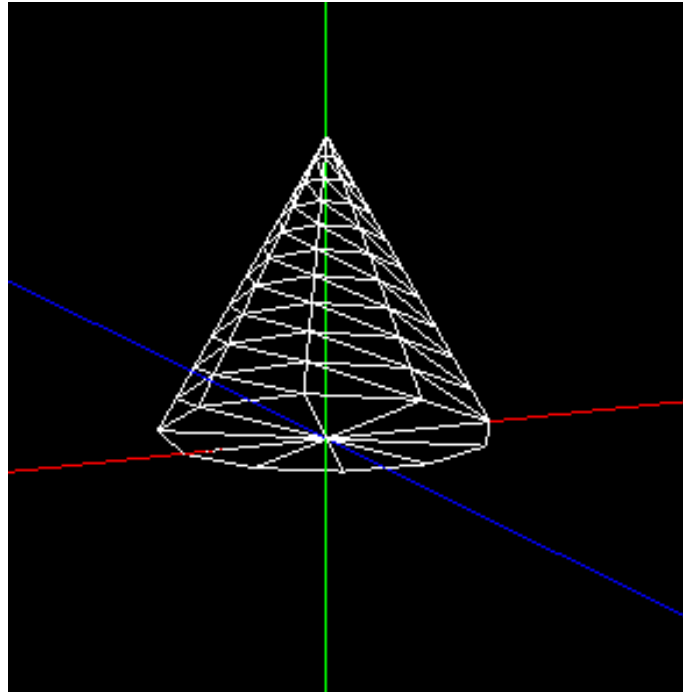


Figura 3.11: Cone com 10 slices e 10 stacks

4 Engine

4.1 Parser

Antes de abordar a **Engine**, é importante destacar o papel do módulo **parser.cpp**. Este módulo é responsável por interpretar ficheiros XML que descrevem as figuras 3D a serem renderizadas. O **parser.cpp** utiliza a biblioteca **tinycl2** para percorrer o ficheiro XML e extrair os seguintes elementos:

- **Modelos 3D**: Ficheiros .3d que contêm a definição dos vértices das figuras.
- **Parâmetros da câmara e projeção**: Definições que influenciam a visualização da figura.

Após o processamento, os dados extraídos são armazenados na estrutura **World**, que será utilizada pelo **engine.cpp** para a renderização da figura.

4.2 Engine

Finalmente, o **engine.cpp** é o módulo que carrega um ficheiro XML, extrai informações sobre a cena e utiliza essa informação para renderizar modelos tridimensionais. A partir dos ficheiros .3d especificados no ficheiro de configuração, a Engine lê os vértices na ordem em que aparecem no ficheiro e desenha diretamente os pontos para a formação de triângulos.

A leitura do ficheiro XML é feita no módulo **parser.cpp**, onde os dados extraídos são armazenados na estrutura de dados **World**, que contém informações sobre a câmara, projeção e modelos da cena.

A estrutura **World** contém as seguintes variáveis:

- **camera.position**: um vetor que define a posição da câmara (x, y, z).
- **camera.lookAt**: um vetor que define a direção para a qual a câmara está orientada.
- **camera.up**: um vetor que define a orientação "para cima" da câmara.
- **camera.projection**: um vetor que contém os parâmetros de projeção fov, near e far.

- **group.models**: uma lista com os nomes dos ficheiros dos modelos 3D a serem carregados e desenhados.

Para manipular a câmara, utilizamos as seguintes variáveis globais, inicializadas a partir das respetivas expressões:

- **radius** = $\sqrt{x^2 + y^2 + z^2}$, que define a distância da câmara ao ponto de referência.
- **beta** = $\sin^{-1}\left(\frac{y}{radius}\right)$, que define o ângulo vertical da câmara.
- **alfa** = $\tan^{-1}\left(\frac{x}{z}\right)$, que define o ângulo horizontal da câmara.

Estas variáveis permitem modificar a vista da câmara, conforme o input do utilizador, permitindo interagir melhor com a figura projetada.

5 Utilização

Para a correta utilização do programa sugerimos a execução do programa em ambiente **Linux**, uma vez que o nosso grupo desenvolveu neste sistema operativo e, portanto, em outros so's o **CMake** não irá funcionar corretamente. Os ficheiros **XML** deverão também estar no pasta **tests**.

Em primeiro lugar devemos utilizar as seguintes pastas no **CMake** e fazer **Configure** e **Generate**:

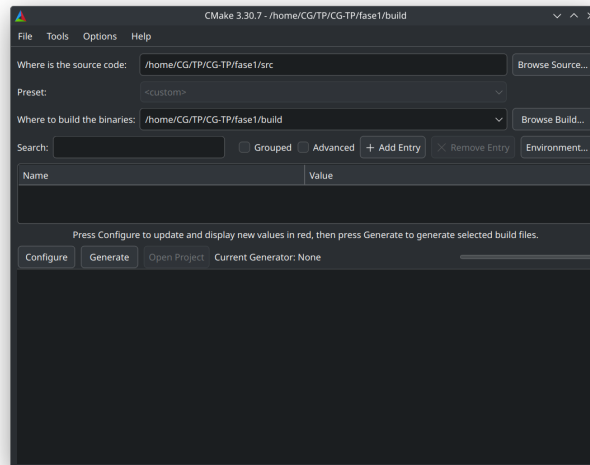
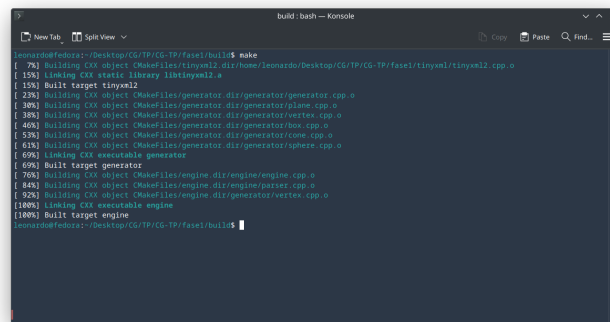


Figura 5.1: CMake

De seguida, devemos fazer **make** na pasta build:



```
leomardofedora:~/Desktop/CG/TP/CG-TP/phase/build$ make
[ 78%] Building CXX object Chakefiles/tinymtl.dir/home/leomardo/Desktop/CG/TP/CG-TP/phase/tinymtl/tinymtl2.cpp.o
[ 15%] Linking CXX static library libtinymtl.a
[ 15%] Built target tinymtl
[ 22%] Building CXX object Chakefiles/generator.dir/generator/generator.cpp.o
[ 30%] Building CXX object Chakefiles/generator.dir/generator/plane.cpp.o
[ 38%] Building CXX object Chakefiles/generator.dir/generator/vertex.cpp.o
[ 46%] Building CXX object Chakefiles/generator.dir/generator/box.cpp.o
[ 53%] Building CXX object Chakefiles/generator.dir/generator/cone.cpp.o
[ 61%] Building CXX object Chakefiles/generator.dir/generator/sphere.cpp.o
[ 69%] Linking CXX executable generator
[ 69%] Built target generator
[ 76%] Building CXX object Chakefiles/engine.dir/engine/engine.cpp.o
[ 84%] Building CXX object Chakefiles/engine.dir/engine/parser.cpp.o
[ 92%] Building CXX object Chakefiles/engine.dir/generator/vertex.cpp.o
[100%] Linking CXX executable engine
[100%] Built target engine
leomardofedora:~/Desktop/CG/TP/CG-TP/phase/build$
```

Figura 5.2: Make

Finalmente, podemos, por exemplo, executar os programas generator e engine das seguintes maneiras que, apesar de serem um input errado, darão contexto sobre que valores podem receber:

- `./generator plane`
- `./engine`

É importante realçar que tanto o nome do ficheiro utilizado no input do generator, como o da engine, devem conter apenas o seu nome, sem qualquer referência ao seu path.

6 Conclusão

Nesta primeira fase do trabalho tivemos que usufruir e conciliar vários conhecimentos adquiridos nas aulas práticas. Foi essencial aprofundar o nosso conhecimento das estruturas a desenhar, em particular a esfera e o cone, e as estratégias concretas para realizar esse desenho.

Cumprimos todos os objetivos requisitados desta fase do trabalho; desenvolvemos o generator e a engine pretendidos e recorremos à biblioteca recomendada, tinyXML.

Tivemos maior dificuldade na representação gráfica do processo de desenho realizado; esta fase do trabalho deixou claro o quão complexo pode ser o desenho de objetos aparentemente simples num plano tridimensional quando somos nós a definir as fórmulas e funções que o vão fazer.